

### 3. API Gateway для DSP-интеграции

Интеграция с новой DSP-площадкой требует надёжного, безопасного и производительного API-слоя, который обеспечит приём до 18 000 дополнительных RPS (первоначально) и соблюдение жёсткого SLA по latency (<80 мс). Основой этого слоя является **API Gateway (реализация возможна с помощью NGINX или Envoy Proxy)**, который выступает единой точкой входа, а также реализует ряд функций безопасности, отказоустойчивости и управления запросами.

#### 3.1 . Маршрутизация запросов на ставки

- Все внешние RTB-запросы поступают на единый эндпоинт (API Gateway);
- API Gateway маршрутизирует запросы в сервис ставок;
- Для новой DSP-площадки может потребоваться специфическая обработка (например, дополнительная валидация, трансляция полей OpenRTB).

#### 3.2 Ограничение частоты запросов (Rate Limiting)

Для защиты системы от перегрузки и соблюдения SLA, необходимо ограничить количество запросов от каждого DSP-партнёра.

##### Реализация:

- Используем функцию API GW Rate - Limit Filter;
- Настраиваем лимиты как для всей платформы (глобальный) и для DSP (по X-DSP-Id);
- Для нового DSP: начальный лимит 20 процентов от максимального RPS (с возможностью повышения);
- При превышении лимита Gateway возвращает HTTP 429 (Too Many Requests) без передачи запроса в бэкенд.

**Дополнительно** можно применить сглаживание пиков запросов / трафика (token bucket).

#### 3.3. Аутентификация

Для новой DSP-площадки выбираем **mTLS (Mutual TLS)** как основной метод аутентификации, так как он обеспечивает высокий уровень безопасности и не добавляет заметной задержки.

#### **Обоснование:**

- mTLS проверяет как подлинность клиента (сертификат), так и сервера, что исключает подделку запросов.
- Не требует передачи секретов в каждом запросе (в отличие от API-ключей или JWT, которые нужно валидировать и парсить).
- Вписывается в инфраструктуру с использованием API GW, который поддерживает mTLS из коробки (через SDS для сертификатов).

### **3.4. Паттерн автоматического размыкателя (Circuit Breaker)**

Circuit Breaker применяется как к вызовам внутренних зависимостей на уровне API-шлюза (API Gateway) так и со стороны внешнего партнёра (например, если DSP начинает отправлять слишком много запросов, превышая лимиты, или при аномальном поведении). Таким образом, применяя данный паттерн мы не усиливаем каскадные отказы и снижаем вероятность «положить» сервис ставок.

### **3.5. Мониторинг времени отклика**

Для соблюдения SLA ( $P95 < 80$  мс) критично измерять и визуализировать время ответа на каждом этапе.

- API GW экспортирует встроенную статистику
- Возможно использовать стандарт OpenTelemetry для распределённой трассировки каждого запроса (через заголовки `tracetransparent`).
- В сервисы ставок и кампаний также добавляем метрики (время на каждый вызов) и публикуем их.

### **Настройка связки Prometheus / Grafana**

#### **Алерты:**

- Настраиваем алерты в **Prometheus** на превышение  $P95 \text{ latency} > 70$  ms для кластера Bidding Ingress (запас для учёта времени на обработку внутри).

- Отслеживаем процент ошибок (5xx) и количество срабатываний Rate Limit / Circuit Breaker.

**Дашборды** в Grafana отображают:

- Latency по перцентилям (P50, P95, P99) для всего API-слоя и по каждому DSP-партнёру.
- Тренды ошибок и использования лимитов.
- Трассировка отдельных запросов для глубинного анализа задержек.

### 3.6 Итог

Таким образом, API-слой обеспечивает:

- **Безопасность** через mTLS.
- **Защиту от перегрузок** через rate limiting и circuit breaker.
- **Гибкость** маршрутизации для постепенного ввода нового DSP.
- **Наблюдаемость** для подтверждения соответствия SLA.

Все компоненты конфигурируются декларативно через Kubernetes + Envoy и легко масштабируются при росте нагрузки.